tmux

front

Prefix `C-a` vi copy mode mouse on windows and panes from 1 status bar on top 50k lines of history

Sessions

`C-a f`	Sessionizer: fuzzy-pick a `~/projects` directory, create or switch to its session
`tmux new -s name`	New session
`tmux ls`	List sessions
`tmux attach -t name`	Attach
`tmux kill-session -t name`	Kill
`C-a d`	Detach
`C-a s`	Session picker
`C-a $`	Rename session
`C-a ( / )`	Prev / next session
`C-a L`	Last session

Windows

`C-a c`	New window
`C-a ,`	Rename window
`C-a &`	Kill window
`C-a p / n`	Prev / next window
`C-a 1-9`	Jump to window N
`C-a w`	Window picker
`C-a < / >`	Move window left / right
`C-a !`	Break pane out to a window

Copy Mode (vi)

`C-a [`	Enter copy mode
`v`	Begin selection
`y`	Yank to system clipboard
`/ ?`	Search forward / backward
`o`	Open selection (URL, file)
`C-o`	Open selection in `$EDITOR`
`S`	Search it on DuckDuckGo
`q / Esc`	Exit copy mode

Panes

`C-a |`	Split vertical
`C-a -`	Split horizontal
`C-a \ / _`	Full-width / full-height split
`C-h/j/k/l`	Move; no prefix, passes into nvim
`M-h/j/k/l`	Resize by 3; no prefix, passes into nvim
`C-a H/J/K/L`	Resize by 5
`C-a z`	Zoom / unzoom
`C-a x`	Kill pane
`C-a { / }`	Swap pane left / right
`C-a q`	Show pane numbers
`C-a Space`	Cycle layouts

PLUGINS (TPM)

`C-a C-s / C-r`	Save / restore sessions
`C-a P`	Toggle pane logging
`C-a M-p`	Screen capture to a file
`C-a M-P`	Save the whole pane history

resurrect and continuum: the last save is restored when the server starts. pain-control, yank, open and sensible provide the keys on the front. TPM installs itself on first start.

NVIM BRIDGE

`C-h/j/k/l` and `M-h/j/k/l` are bound without the prefix. smart-splits sets `@pane-is-vim` on a pane running nvim; tmux forwards the key there and nvim moves between its own splits, or hands back to tmux at the edge.

GHOSTTY

Option is Alt, so `M-h` resizes. Ghostty splits and tabs (`Cmd-d`, `Cmd-t`) are unbound; tmux owns layout. Copy on select. Atkinson Hyperlegible Mono 16pt.

COMMAND MODE

`C-a :`	Enter command mode
`:resize-pane -D 10`	Resize down 10 rows
`:swap-window -t 3`	Move window to position 3
`:move-pane -t` *`sess`*`:`	Move pane to another session
`:setw synchronize-panes`	Type in all panes at once
`:capture-pane -S -100`	Capture the last 100 lines
`:rename-window` *`name`*	Rename window

MAINTENANCE

`tmux source ~/.tmux.conf`	Reload config
`C-a I`	Install plugins
`C-a U`	Update plugins
`C-a M-u`	Remove plugins no longer in the config
`tmux kill-server`	Reset everything
`tmux list-keys | grep` *`key`*	Find a binding

nvim: core editing

front

Leader `Space` relative numbers scrolloff 8 system clipboard 4-space indent persistent undo cwd follows the project root

MOTION

`h j k l`	Left, down, up, right
`w / b`	Next / prev word start
`e / ge`	Next / prev word end
`0 / $`	Line start / end
`^`	First non-blank
`gg / G`	File start / end
`{ / }`	Paragraph up / down
`%`	Matching bracket
`fc / Fc`	Find char fwd / back
`; / ,`	Repeat find fwd / back
`C-d / C-u`	Half page down / up, centred
`n / N`	Next / prev match, centred
`Esc`	Clear search highlight

EDITING

`i / a`	Insert before / after
`I / A`	Insert at line start / end
`o / O`	New line below / above
`x`	Delete char
`dd / cc`	Delete / change line
`dw / cw`	Delete / change word
`D / C`	Delete / change to end of line
`rc`	Replace char
`~`	Toggle case
`.`	Repeat last change
`u / C-r`	Undo / redo

VISUAL MODE

`v / V / C-v`	Char / line / block
`< / >`	Indent (stays in visual)
`J / K`	Move selection down / up
`SPC p`	Paste without yanking

TEXT OBJECTS (`d/c/y/v` + object)

`iw / aw`	word
`ip / ap`	paragraph
`i" / a"`	quoted string (also `' ``)
`i( / a(`	brackets (also `[ { <`)
`it / at`	HTML tag

mini.ai adds these, looking up to 500 lines away:

`if / af`	function call
`ia / aa`	function argument
`iq / aq`	any quote
`ib / ab`	any bracket
`in( / il(`	next / last one of that kind

MINI.SURROUND

`sa`*`motionchar`*	Add surround
`sd`*`char`*	Delete surround
`sr`*`oldnew`*	Replace surround

`saiw"` quotes the word; `sr)]` swaps parens for brackets.

nvim: core editing

back

Registers & Macros

`"ay / "ap`	Yank / paste register `a`
`"+y / "+p`	System clipboard (the default)
`:reg`	Show all registers
`qa`	Record macro into `a`
`q`	Stop recording
`@a`	Play macro `a`
`@@`	Repeat last macro
`10@a`	Play macro 10 times

Marks

`ma`	Set mark `a`
`'a`	Jump to exact position
`'a`	Jump to line
`'.`	Last edit position
`''`	Previous jump position

Command Mode

`:s/old/new/g`	Substitute in line
`:%s/old/new/gc`	Substitute in file, confirm
`:g/pat/d`	Delete matching lines
`:r !cmd`	Insert command output
`:!cmd`	Run shell command
`:norm @a`	Macro over the selection

Folds (tree-sitter, open on load)

`za`	Toggle fold
`zc / zo`	Close / open fold
`zM / zR`	Close / open all
`zj / zk`	Next / prev fold

Splits & Buffers

`SPC |`	Vertical split
`SPC -`	Horizontal split
`C-w =`	Equalize splits
`C-w q`	Close split
`C-w o`	Only this split
`C-h/j/k/l`	Move; continues into tmux
`A-h/j/k/l`	Resize; continues into tmux
`S-h / S-l`	Prev / next buffer
`SPC bd`	Delete buffer

Save, Quit, Windows

`SPC w`	Save
`SPC q`	Quit
`SPC Q`	Quit all without saving
`SPC tz`	Zen mode (90 columns)
`SPC tt`	Terminal in this window
`Esc Esc`	Terminal: back to normal mode

A single `Esc` reaches the program inside the terminal.

File Explorer (netrw)

`SPC e / E`	Explorer here / in a split
`CR`	Open file / dir
`-`	Go up
`%`	New file
`d`	New directory
`D`	Delete
`R`	Rename
`SPC as`	Add this file to Claude

nvim: lsp, search & git

front

Servers: clangd, pyright, jdtls, ts_ls, bashls, lua_ls, yamlls, terraformls, dockerls, taplo, julials completion blink.cmp Java starts from `ftplugin/java.lua`

LSP

`gd / gD`	Definition / declaration
`grr`	References
`gri / grt`	Implementation / type definition
`grn / gra`	Rename / code action
`K`	Hover docs
`[d / ]d`	Prev / next diagnostic
`C-w d`	Diagnostic float
`SPC rn / ca / d`	Same three, under the leader
`SPC th`	Toggle inlay hints
`SPC f`	Format buffer

Completion (blink.cmp)

`C-Space`	Open menu / toggle docs
`C-n / C-p`	Next / prev item
`C-y`	Accept
`C-e`	Dismiss
`C-f / C-b`	Scroll docs
`C-k`	Toggle signature help

Git

`SPC gg`	lazygit
`SPC gs`	Status picker
`SPC gc`	Commits picker
`SPC gf`	This file's history

Gitsigns (hunks)

`]h / [h`	Next / prev hunk
`SPC hs`	Stage hunk
`SPC hr`	Reset hunk
`SPC hb`	Blame line
`SPC hd`	Diff against index
`SPC hD`	Diff against last commit

Telescope

`SPC sf`	Find files
`SPC sg`	Live grep
`SPC sw`	Grep word under cursor
`SPC sb`	Buffers
`SPC s.`	Recent files
`SPC s?`	Help tags
`SPC sk`	Keymaps
`SPC sd`	Diagnostics
`SPC sr`	Resume last picker
`SPC sn`	This nvim config
`SPC sN / sO`	Grep / browse `~/notes`
`SPC /`	Fuzzy find in buffer

Inside a picker: `C-n/p` move, `CR` open, `C-x` split, `C-v` vsplit, `C-q` to quickfix, `C-u/d` scroll preview, `Esc` close.

Debug (nvim-dap)

`SPC Db / DB`	Toggle / conditional breakpoint
`SPC Dc`	Continue, or start (pick a config)
`SPC Do / Di / DO`	Step over / into / out
`SPC Dr / Dl / Dt`	REPL / run last / terminate
`SPC Du`	Toggle dap-ui

debugpy (per project: `uv add -dev debugpy`), lldb-dap (C/C++), pwa-node (JS/TS), java (jdtls offers the main classes). `make debug-adapters` fetches the Java and JS adapters.

nvim: lsp, search & git

back

Claude Code (claudecode.nvim)

`SPC ac / af`	Toggle / focus the Claude split
`SPC ar / aC`	Resume a session / continue the last
`SPC am`	Select model
`SPC ab`	Add this buffer to the context
`SPC as`	Send selection; in netrw, add the file
`SPC aa / ad`	Accept / deny the open diff

Edits open as a vertical diff: change the right side if you like, then `:w` accepts and `:q` rejects. One Claude per nvim. `:checkhealth claudecode` if the CLI is not found.

Lint (nvim-lint)

`SPC ll` Lint now

Runs on open, save and leaving insert mode. ruff (Python), tflint (Terraform). Other diagnostics come from the LSP.

Formatting (conform.nvim)

On save, same table as Emacs: stylua (Lua), ruff (Python), prettier (JS/TS, JSON, CSS, HTML, YAML, Markdown), terraform fmt, taplo (TOML). C/C++, Java and shell only trim whitespace; Quarto is left alone. `SPC f` formats by hand.

Tree-sitter

Parsers compile with the tree-sitter CLI on first start: bash, c, cpp, css, diff, dockerfile, gitcommit, git_rebase, hcl, html, java, javascript, json, julia, lua, luadoc, make, markdown, markdown_inline, python, query, sql, ssh_config, terraform, toml, tsx, typescript, vim, vimdoc, yaml. Highlighting, folds and indentation (not Python or YAML) start for any filetype with a parser. A sticky header shows the enclosing function or class.

Plugins (vim.pack)

`:lua vim.pack.update()`	Fetch updates; `:w` in the diff buffer applies them
`:lua vim.pack.update(nil, { target = "lockfile" })`	Revert to `nvim-pack-lock.json`
`:lua vim.pack.del({ "name" })`	Remove a plugin from disk
`:checkhealth vim.pack`	Plugin health

Add or remove plugins in the `require("kwb.pack")` lists, restart, commit the lockfile.

Tree-sitter Commands & Health

`:TSInstall lang`	Compile one parser
`:TSInstall! lang`	Reinstall, relinking its queries
`:checkhealth nvim-treesitter`	Why highlighting is off
`:checkhealth vim.lsp`	Servers; `:LspInfo` for this buffer
`:checkhealth`	Everything
`:restart`	Restart nvim in place

emacs

Command is Meta right Option is Control `Esc` quits like `C-g` `TAB` indents, then completes which-key after half a second `y/n` answers

Files & Buffers

`C-x C-f`	Find file
`C-x C-s`	Save
`C-x b`	Switch buffer (recent files too)
`C-x 4 b`	Switch in other window
`C-x p b`	Project buffers
`C-x C-b`	ibuffer
`C-x k`	Kill buffer
`C-x r b`	Bookmarks
`C-x u`	Undo tree (vundo)
`C-/`	Undo

Windows

`C-x 2 / 3`	Split below / right
`C-x 0 / 1`	Close this / all others
`M-o`	Jump to window by letter
`C-x o`	Next window (repeat `o`)

Search & Jump

`C-s / C-r`	Incremental search (shows 3/17)
`M-%`	Query replace
`M-s r`	ripgrep the project
`M-s l`	Lines in this buffer
`M-g g`	Go to line, with preview
`M-g i`	imenu (symbols)
`M-g f`	Flymake diagnostics
`M-g n / p`	Next / prev diagnostic (repeats)

Completion (corfu)

`C-n / C-p`	Move (opens after 2 chars)
`RET / TAB`	Accept
`C-g`	Dismiss

Editing

`M-n / M-p`	Open a line below / above
`M-w / C-w`	Copy / kill region
`C-y / M-y`	Yank / cycle kill ring
`C-k`	Kill to end of line
`M-;`	Comment dwim
`C-x C-;`	Comment line
`M-q`	Refill paragraph
`C-> / C-<`	Cursor at next / prev match
`C-c C-<`	Cursor at every match
`C-S-c C-S-c`	Cursor on each line of region

Pairs insert themselves; camelCase words split in code.

Minibuffer (vertico, orderless, embark)

`foo bar`	Space-separated terms, any order
`C-n / C-p`	Move (wraps)
`RET`	Choose
`M-RET`	Submit what you typed
`TAB`	Insert candidate
`C-.`	Act on candidate (embark)
`C-;`	Default action (embark)
`C-h B`	Bindings in this context

Project (`C-c p`) & Tools

`C-c p f / g`	Find file / grep in project
`C-c p p / b`	Switch project / buffer
`C-c p s / e`	Shell / eshell
`C-c p c / d`	Compile / dired
`C-c g`	magit
`C-c t`	vterm
`C-c C-'`	Claude Code menu

LSP (EGLOT)

`M-.` / `M-,`	Definition / back
`M-?`	References
`C-c l r`	Rename
`C-c l a`	Code actions
`C-c l f`	Format
`C-c l d` / `h`	Doc at point / doc buffer

Starts itself for C, Python, Java, JS/TS, Julia, shell, YAML, Dockerfile, Lua. Inlay hints are opt-in (below).

DEBUG (dape, `C-c d`; keys repeat)

`d`	Start (choose a config)
`b` / `B` / `e`	Breakpoint / clear all / expression
`n` / `s` / `o`	Next / step in / step out
`c` / `p`	Continue / pause
`r` / `q` / `D`	Restart / quit / disconnect
`i` / `R` / `w` / `x`	Info / REPL / watch / evaluate

Same adapters as nvim: debugpy, lldb-dap, js-debug, jdtls.

MAGIT (`C-c g`)

`TAB`	Expand section
`s` / `u`	Stage / unstage hunk at point
`S` / `U`	Stage / unstage all
`c c`	Commit, then `C-c C-c`
`P p` / `F p`	Push / pull
`b b` / `b c`	Checkout / new branch
`l l` / `d d`	Log / diff
`z z`	Stash
`r i`	Interactive rebase
`k`	Discard
`g` / `q` / `?`	Refresh / quit / help

Change bars in the fringe (diff-hl). `M-x git-timemachine` walks a file's history.

CLAUDE CODE (`C-c C-'`)

The menu starts or stops Claude, sends a prompt, continues or resumes a session, toggles the window, lists sessions. Claude sees the selection, xref, diagnostics and imenu. Edits open in ediff side by side: `n`/`p` step hunks, edit buffer B, then accept or reject at the prompt. One Claude per project. Files it edits reload on their own.

TERMINAL (VTERM)

`C-c t`	Open
`C-c C-t`	Copy mode (normal Emacs keys)
`C-c C-c`	Send `C-c` to the shell

ORG (`~/org`)

`C-c a` / `c` / `l`	Agenda / capture / store link
`TAB` / `S-TAB`	Cycle heading / whole file
`M-RET`	New heading
`C-c C-t`	Cycle TODO (done is timestamped)
`C-c C-s` / `C-d`	Schedule / deadline
`M-up` / `down`	Move subtree
`C-c C-c`	Tags; toggle checkbox
`<s TAB`	Source block; `C-c '` edits it

JULIA

`C-c C-z`	REPL (in vterm)
`C-c C-c`	Send region

MAINTENANCE (`M-x`)

`package-upgrade-all`	Update packages
`package-vc-upgrade`	Update claude-code-ide (from GitHub)
`package-autoremove`	Drop packages no longer in the config
`apheleia-mode`	Toggle format on save (same table as nvim)
`eglot-inlay-hints-mode`	Inlay hints; `eglot-reconnect` when it wedges
`treesit-install-language-grammar`	Missing grammar (needs git and clang)
`modus-themes-toggle`	Dark for a day

git

front

Pager delta, side by side pull rebases rebase auto-stashes and auto-squashes rerere remembers resolutions fetch prunes push follows tags conflicts zdiff3

Your Aliases

`st`	status	`br`	branch
`sts`	status –short	`ci`	commit
`co`	checkout	`prev`	last branch
`hist`	log graph	`unstage`	restore –staged
`amend`	amend, no edit	`undo`	soft reset HEAD~1

Basics

`git init`	New repo on `main`
`git clone url`	Clone
`git add file`	Stage file
`git add -p`	Stage by hunk
`git ci -m "msg"`	Commit
`git push -u origin b`	Push new branch
`git pull`	Fetch and rebase

Branching

`git br`	List branches
`git switch -c feat`	New branch
`git switch main`	Switch to main
`git prev`	Toggle last branch
`git br -d b`	Delete merged
`git br -D b`	Force delete

delta

`n / N`	Next / prev file
`/`	Search
`q`	Quit
`delta f1 f2`	Diff two files

Line numbers on, ANSI theme, light background.

Inspecting

`git st`	Status
`git diff`	Unstaged (delta)
`git diff -staged`	Staged changes
`git hist`	Pretty log graph
`git log -oneline -20`	Last 20 commits
`git show abc123`	Show a commit
`git blame file`	Line authorship
`git log -p - file`	File history

Stashing

`git stash`	Stash changes
`git stash pop`	Restore and drop
`git stash list`	List stashes
`git stash show -p`	Show contents
`git stash drop`	Delete top stash

Rebase

`git rebase main`	Rebase onto main
`git rebase -i HEAD~3`	Edit the last 3 commits
`git rebase -continue`	After resolving
`git rebase -abort`	Cancel rebase
`git ci -fixup abc`	Fixup; squashed on rebase

A dirty worktree is fine: rebase stashes and restores.

Undoing

`git unstage file`	Unstage (alias)
`git undo`	Uncommit, keep staged (alias)
`git amend`	Amend (alias)
`git restore file`	Discard unstaged changes
`git revert abc`	Revert commit
`git reset -soft HEAD~3`	Undo 3, keep staged
`git reset HEAD~1`	Undo and unstage
`git reflog`	Find lost commits

Merge Conflicts (zdiff3)

`<<<<< HEAD`	Your side
`|||||||`	Common ancestor
`=======`	Their side
`>>>>> branch`	End marker

rerere replays a resolution the next time the same conflict appears.

GitHub CLI (gh)

`gh pr create`	Create PR
`gh pr view -web`	Open current PR
`gh pr list`	List open PRs
`gh pr checkout 123`	Check out PR
`gh pr checks`	CI status
`gh issue list`	List issues
`gh issue create`	Create issue
`gh repo clone u/r`	Clone repo
`gh browse`	Open in browser

Cherry-Pick & Bisect

`git cherry-pick abc`	Apply commit here
`git bisect start`	Binary search for a bug
`git bisect bad`	Mark current bad
`git bisect good abc`	Mark commit good
`git bisect reset`	Done bisecting

Config

`~/.config/git/config` is shared; email and credential helper live in `~/.gitconfig-local`. Histogram diff, moved lines coloured, LFS filters on, git-secrets and git-filter-repo installed.

Useful Patterns

`git log -all -oneline -graph`	Branch topology
`git diff main...HEAD`	What this branch changed
`git log main..HEAD`	Commits since branching
`git log -S`*`str`*` -p`	Commits that add or remove *str*
`git worktree add ../x br`	Second checkout of the repo

cli tools

front

Aliases: `ll`, `lt` (eza), `vi`/`vim` (nvim), `bc` (`bc -l`). `ls`, `cat`, `grep` and `find` stay stock; replacements keep their own names.

BAT

`bat file.py`	Syntax, line numbers
`bat -l json`	Force language
`bat -p file`	Plain output
`bat -A file`	Show non-printable
`bat -diff file`	Git diff inline
`bat -r 10:20 file`	Lines 10–20 only

Also the man pager. ANSI theme.

EZA

`ll`	`eza -la -git -icons`
`lt`	`eza -tree -level=2 -icons`
`eza -l -s modified`	Sort by time
`eza -l -s size`	Sort by size
`eza -git-ignore`	Respect .gitignore

FZF

`C-t`	Pick a file into the line
`M-c`	cd into a directory
`C-r`	History search
`vim $(fzf)`	Open a file
`rg pat | fzf`	Filter grep results

fd lists files and dirs for `C-t` and `M-c`, rg for bare `fzf`; hidden files in, `.git` out. Inside: `C-j/k` move, `Tab` multi-select.

DUST (du)

`dust`	Largest dirs, as a tree
`dust -d 2 path`	Two levels deep
`dust -n 10`	Top 10 only

FD

`fd pattern`	Find files
`fd -e py`	By extension
`fd -t d / -t f`	Dirs / files only
`fd -H`	Include hidden
`fd -I`	Ignore .gitignore
`fd pat -x cmd {}`	Run on each
`fd -changed-within 1d`	Recent

RIPGREP

`rg pattern`	Search recursively
`rg pat -t py`	Python files only
`rg pat -g '*.yaml'`	Glob filter
`rg -i pattern`	Case insensitive
`rg -w pattern`	Whole word
`rg -l pattern`	File names only
`rg -c pattern`	Count per file
`rg -C 3 pattern`	3 lines of context
`rg -no-ignore pat`	Include gitignored
`rg -F 'literal'`	Fixed string
`rg pat -r repl`	Show replacement

nvim's `:grep` uses rg too.

cli tools

back

JQ

`jq '.'`	Pretty-print
`jq '.key'`	Extract key
`jq '.items[0]'`	First element
`jq '.items[].name'`	Map over array
`jq 'select(.n > 3)'`	Filter
`jq -r '.name'`	Raw (no quotes)
`jq -c '.'`	Compact output
`jq 'keys'`	All keys
`jq 'length'`	Count items
`jq -s '.'`	Slurp a stream into an array

XH (curl, HTTPie syntax)

`xh url`	GET
`xh POST url k=v`	POST JSON
`xh POST url k:=123`	Non-string value
`xh -f POST url k=v`	Form POST
`xh url Header:val`	Custom header
`xh -d url`	Download
`xh -offline ...`	Preview the request

ZOXIDE

`z dotfiles`	Jump to best match
`z dot`	Partial match
`zi`	Interactive (fzf)
`z -`	Previous directory

DIRENV

`direnv allow`	Trust this `.envrc`
`direnv deny`	Revoke trust

Loads on `cd` in, unloads on `cd` out. Both editors pick it up; dap resolves `.venv` at launch.

TLDR (tealdeer)

`tldr tar`	Quick examples
`tldr -update`	Refresh the cache

HYPERFINE

`hyperfine 'cmd'`	Benchmark with stats
`hyperfine 'a' 'b'`	Compare two commands
`hyperfine -w 3 'c'`	3 warm-up runs
`-export-markdown f`	Results as a table

UV

`uv sync`	Install project deps
`uv run` *`cmd`*	Run inside the venv
`uv add -dev` *`pkg`*	Add a dev dependency
`uv tool install` *`p`*	Global CLI (ipython, visidata)
`uvx` *`cmd`*	Run a tool without installing

MAKE (dotfiles)

`make`	Stow all configs
`make doctor`	Check links, tools, editor toolchains, Brewfile
`make lint`	shellcheck, stylua, nvim syntax, elisp parens
`make homebrew`	Install the Brewfile
`make brewfile`	Dump installed state to the Brewfile
`make tools`	npm globals (LSPs, prettier) and uv tools
`make debug-adapters`	Java and JS debug adapters for both editors
`make sync`	Pull, brew, stow
`make cards`	Build these PDFs
`make ssh`	Deploy SSH config
`make keyboard`	Reload the key remap agent

LAZYGIT (`SPC gg` in nvim)

Panels: 1 Status, 2 Files, 3 Branches, 4 Commits, 5 Stash. `Tab` or the number moves between them.

`Space`	Files: stage / unstage
`a`	Files: stage all
`Enter`	Files: stage by hunk or line
`d`	Files: discard
`c / C`	Commit / commit in editor
`A`	Amend last commit
`P / p`	Push / pull
`Space`	Branches: checkout
`n`	Branches: new branch
`r / M`	Branches: rebase onto / merge into current
`s / f`	Commits: squash / fixup into the one below
`r`	Commits: reword
`e`	Commits: edit (interactive rebase)
`d`	Commits: drop
`} / {`	More / less diff context
`+ / _`	Bigger / smaller panel
`/`	Search the panel
`? / q`	Help / quit

Theme follows the terminal palette; selection is bold, not a bar.

YAZI

`yy` opens it and changes directory on exit; `yazi` *`path`* opens at a path. Hidden files shown, sizes in the list.

`h / l`	Parent / enter directory
`j / k`	Down / up
`Enter / o`	Open
`O`	Open with (choose program)
`Space`	Toggle select
`v / C-a`	Visual mode / select all
`y / x / p`	Yank / cut / paste
`d / D`	Trash / delete permanently
`a`	Create (end with `/` for a dir)
`r`	Rename
`/`	Find here (`n / N` next / prev)
`s / S`	Search names (fd) / contents (rg)
`z / Z`	Jump with zoxide / fzf
`g h`	Home
`.`	Toggle hidden
`Tab`	Info about the hovered file
`t`	New tab (`[ ]` to switch)
`~`	Help
`q`	Quit

tui tools

back

GLOW (markdown)

`glow file.md`	Render a file
`glow`	Browse markdown files here
`glow -p file.md`	Pager mode
`glow -w 80 file.md`	Set width

LAZYDOCKER

`Enter`	Logs / detail for the item
`e`	Exec a shell in the container
`s`	Stop
`r`	Restart
`d`	Remove
`a`	Attach
`[ / ]`	Prev / next tab
`/`	Filter
`q`	Quit

DIVE (image layers)

`dive` *`image`*	Explore layers and files
`Tab`	Switch layers / files pane
`C-u`	Hide unchanged files
`C-f`	Filter files
`C-c`	Quit

SHELLCHECK

`shellcheck script.sh`	Lint
`shellcheck -e SC2086`	Exclude a rule
`shellcheck -f diff`	Patch of auto-fixes
`shellcheck -s bash`	Force dialect
`shellcheck -x`	Follow `source`

Common: SC2086 unquoted variable, SC2046 unquoted command substitution, SC2155 declare and assign in one line. `make lint` runs it on the bash files.

DUCKDB

`duckdb`	SQL shell, in memory
`duckdb db.duckdb`	Persistent database
`FROM 'f.parquet'`	Query a file directly
`.read f.sql`	Run a script
`.mode markdown`	Output format

VISIDATA

`vd file.csv`	Open (CSV, Parquet, JSON, SQLite)
`Shift-F`	Frequency table of the column
`[ / ]`	Sort ascending / descending
`/`	Search the column
`C-s`	Save sheet
`q`	Close sheet

zsh & shell

zsh starship prompt autosuggestions and syntax highlighting spelling correction asks before running emacs keymap

Line Editing

`C-a / C-e`	Line start / end
`M-b / M-f`	Word back / forward
`C-w`	Delete word backward
`M-d`	Delete word forward
`C-u`	Delete to line start
`C-k`	Delete to line end
`C-y`	Paste deleted text
`C-r`	History search (fzf)
`C-t`	File picker (fzf)
`M-c`	Fuzzy cd (fzf)
`Tab`	Complete (menu; hidden files too)
`→`	Accept suggestion
`M-f`	Accept next word of it

History (8192 lines, shared live)

`!!`	Repeat last command
`!$`	Last argument
`!abc`	Last command starting `abc`
`!abc:p`	Print, do not run
` cmd`	Leading space: not saved
`history | rg pat`	Search history

Job Control

`C-z`	Suspend foreground
`bg / fg`	Resume in background / foreground
`jobs`	List jobs
`kill %1`	Kill job 1
`cmd &`	Start in background

Redirection & Piping

`c1 && c2`	Run c2 if c1 succeeds
`c1 || c2`	Run c2 if c1 fails
`c1 | c2`	Pipe stdout
`c1 |& c2`	Pipe stdout and stderr
`cmd > file`	Redirect stdout
`cmd 2>&1`	Stderr to stdout
`cmd &> file`	Redirect both
`cmd » file`	Append
`cmd < file`	Stdin from file
`cmd «< "str"`	Stdin from a string
`$(cmd)`	Substitute output

Dotfiles Functions

`note`	Today's note in `~/notes`, synced to git
`notes`	Grep the notes in nvim
`yy`	yazi, then `cd` to where it ended
`C-a f`	tmux sessionizer over `~/projects`
`nvm / node`	Load nvm on first use (npm, npx too)

Prompt (starship)

Directory (3 segments, cut at the repo), branch, red status flags, `(venv)`, and the last command's time when over 2 s. A red `>` means the last command failed.

Keyboard

Right Option is Control (hidutil launch agent; `make keyboard` reloads it). In Emacs, Command is Meta.

zsh & shell

back

Zsh Globbing

`**/*.py`	Recursive glob
`*.txt~R*`	Exclude pattern
`*(.)`	Regular files only
`*(/)`	Directories only
`*(om[1,5])`	5 most recently modified
`*(Lk+100)`	Files over 100 KB
`*(@)`	Symlinks only
`*(N)`	No error when nothing matches

Parameter Expansion

`${v:-default}`	Default if unset
`${v:=default}`	Assign if unset
`${v#pat}`	Remove shortest prefix
`${v##pat}`	Remove longest prefix
`${v%pat}`	Remove shortest suffix
`${v%%pat}`	Remove longest suffix
`${v/old/new}`	Replace first
`${v//old/new}`	Replace all
`${v:u} / ${v:l}`	Upper / lower (zsh)
`${#v}`	Length
`${v:t} / ${v:h}`	Basename / dirname (zsh)

Builtins

`type cmd`	What is this?
`which cmd`	Path to command
`where cmd`	All locations
`whence -v cmd`	Verbose type
`command cmd`	Bypass aliases
`source file`	Run in this shell
`exec cmd`	Replace the shell

Environment

`env`	Show all vars
`export K=v`	Set env var
`unset K`	Remove var
`printenv K`	Print one var
`echo $PATH | tr : '\n'`	PATH, one per line

Shell Layering

`~/.profile`	Login env: PATH, Homebrew, Rust, Java, TeX
`~/.shell-common`	Aliases, fzf vars, nvm lazy-load, `yy`
`~/.zshrc`	Completions, plugins, prompt, `note`
`~/.bashrc`	Same for bash (Linux boxes)

`time zsh -i -c exit` measures startup; zprof lines are in `.zshrc`. compinit rebuilds its dump once a day.

Quick Reference

`ls -la` → `ll`
`tree` → `lt`
`cd` → `z` (learns)
`grep -r` → `rg`
`find` → `fd`
`du` → `dust`
`curl` → `xh`
`man` → `tldr` for examples
`time` → `hyperfine` for benchmarks